Module Guide for Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

November 14, 2026

1 Revision History

Date	Version	Notes
Nov. 12, 2026	-1.0	Added Revision -1 content: LC, UC, Modules, SRS Traceability
Nov. 13, 2026	-1.1	Addressed review for Rev -1 content

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
FR	Functional Requirement
Keystone	Platform for capstone team formation and project selection
LC	Likely Change (from the SRS)
M	Module
MG	Module Guide
MGUC	Module Guide Unlikely Change
NFR	Non-Functional Requirement
OS	Operating System
SRS	Software Requirements Specification
SSO	Single Sign-On
TA	Teaching Assistant
UI	User Interface

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms . . . ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes . . . 2
4.2 Unlikely Changes . . . 3

5 Module Hierarchy **4**

6 Connection Between Requirements and Design **6**

7 Module Decomposition **7**
7.1 Hardware-Hiding Module . . . 7
7.2 Behaviour-Hiding Module . . . 7
7.2.1 User Interface Module . . . 7
7.2.2 Authentication Module . . . 8
7.2.3 Authorization Module . . . 8
7.2.4 Course Management Module . . . 8
7.2.5 Enrollment Module . . . 8
7.2.6 Projects Module . . . 9
7.2.7 Students and Teams Module . . . 9
7.2.8 Recommendation Module . . . 9
7.2.9 Approval Workflow Module . . . 9
7.2.10 Threads Module . . . 10
7.2.11 Notifications Module . . . 10
7.2.12 Checklist Module . . . 10
7.2.13 Course Report Module . . . 10
7.3 Software Decision Module . . . 11
7.3.1 Persistence Module . . . 11
7.3.2 Audit Log Module . . . 11

8 Traceability Matrix **11**

9 Use Hierarchy Between Modules **14**

10 User Interfaces **15**

11 Design of Communication Protocols **16**

12 Database Design **16**

13 Timeline **16**

14 Generative AI Use Disclosure **16**
14.1 AI Tools Used 16
14.2 How and Where Used 16
14.3 Verification of AI-Generated Content 16

List of Tables

1 Module Hierarchy 5
2 Trace Between Functional Requirements and Modules 13
3 Trace Between Non-Functional Requirements and Modules 14
4 Trace Between Anticipated Changes and Modules 14

List of Figures

1 Use hierarchy among modules 15

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in Keystone, since each course may need different rules and processes.

Our design follows the rules laid out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes three traceability matrices. Two check the completeness of the design against the functional and non-functional requirements in the SRS. The third shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

The anticipated changes below are a superset of the likely changes in the SRS (Patel et al., 2026). Anticipated changes sourced directly from the SRS are identified.

AC_Hardware **Hardware Platform.** The operating system, browser, and container runtime on which Keystone is run and accessed may change, for example a new hosting environment or new browser versions (NFR_Browsers, NFR_Container). *Module:* M_HardwareHiding.

AC_UserInterface **User Interface.** The appearance, layout, and navigation of the platform's views may change, for example after usability testing. *Module:* M_UserInterface.

AC_SignInProcedure **Sign-In Procedure.** How a user signs in, including the identity providers used (McMaster SSO and a provider for external supervisors) and the protocols for reaching them, and which actions require the user to re-authenticate, may change. *SRS Likely Change:* LC_SignIn. *Module:* M_Authentication.

AC_RolePermissions **Role Permissions.** What each role may see and do may change, and new roles may be added. *SRS Likely Change:* LC_Users. *Module:* M_Authorization.

AC_CourseConfig **Course Configuration.** The set of course features that can be switched on and off, and the course parameters an instructor can set, may change. *SRS Likely Change:* LC_CourseFeatures. *Module:* M_CourseManagement.

AC_EnrollmentSource **Enrollment Source.** Where enrollment data comes from (Avenue to Learn, an uploaded file), and the format it arrives in, may change. *Module:* M_Enrollment.

AC_ProposalTemplate **Proposal Template.** The fields of a project proposal, and which of them are required or optional, may change and may differ between courses. *SRS Likely Change:* LC_ProposalTemplates. *Module:* M_Projects.

AC_TeamFormation **Team Formation.** How students form teams may change and may differ between courses. This includes the information collected in student and team profiles, and the rules a team must satisfy in a course (size limits, permitted program

combinations, eligibility of students from other departments). *SRS Likely Changes:* LC_ProfileFields, LC_TeamCourseRestrictions. *Module:* M_StudentsTeams.

AC_Recommendation **Recommendation Method.** How teammates, teams, and projects are matched may change, be extended to more criteria, or be removed. *SRS Likely Change:* LC_Recommendations. *Module:* M_Recommendation.

AC_ApprovalWorkflow **Approval Process.** The steps, approvers, and mode (in-platform, external, or integrated) by which a course approves team projects may change and may differ between courses. This includes the external system a course uses (e.g.: a GitLab merge request) and its interface. *SRS Likely Change:* LC_ApprovalWorkflows. *Module:* M_ApprovalWorkflow.

AC_ThreadTargets **Discussion Thread Targets.** Which records a discussion thread can be attached to, and how many threads each may have, may change. *SRS Likely Change:* LC_ThreadTargets. *Module:* M_Threads.

AC_NotificationRules **Notification Rules.** Which events cause a notification, who receives it, and how it is delivered may change. *SRS Likely Change:* LC_NotificationRules. *Module:* M_Notifications.

AC_RequiredActions **Required Actions.** Which actions appear on a user's checklist of required actions may change. *SRS Likely Change:* LC_RequiredActions. *Module:* M_Checklist.

AC_Report **Course Reports.** The contents of course reports, and the format they are exported in, may change. *SRS Likely Change:* LC_CourseReports. *Module:* M_CourseReport.

AC_Persistence **Data Storage.** The technology used to store Keystone's records, and how the records are laid out in it, may change. *Module:* M_Persistence.

AC_Audit **Audit Records.** How audit records are stored at rest, including the tamper-evident mechanism, may change. *Module:* M_AuditLog.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

MGUC_CoreRoles **Core Roles.** The four core roles (Student, Supervisor, Instructor, and TA) will not change, since they are the primary users of the system. *SRS Unlikely Change:* Core Stakeholder Roles.

MGUC_HumanApproval **Human Approval.** Approval decisions on proposals and team projects will always be made by a person, never automatically. *SRS Unlikely Change:* Human-centered approval.

MGUC_CourseOrganization **Course Organization.** Students, instructors, TAs, projects, and teams will remain organized by course. *SRS Unlikely Change:* Course Organization.

MGUC_OverallFlow **Overall Flow.** Students may browse projects at any time, with or without a team, and form teams based on their interests. A student must be in a team to apply for or propose a project, and the team's project is then approved. *SRS Unlikely Change:* Vision and Overall Flow.

MGUC_ProjectAssignment **Project Assignment.** A project will move from proposal, to approval, to being open for applications, to being taken by teams. A team will apply to a project and then reach agreement with its supervisor. These states of a project and of an application will not change.

MGUC_IO **Input and Output Devices.** Input will come from users through a web browser (keyboard, pointer, and uploaded files), and output will go to the browser screen and to email.

MGUC_WebDelivery **Web Delivery.** Keystone will be delivered as a web application, with a user interface in the browser and the rest of the system on a server.

MGUC_SingleInstitution **Single Institution.** Each deployment of Keystone will serve one institution.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M_HardwareHiding: Hardware-Hiding Module

M_UserInterface: User Interface Module

M_Authentication: Authentication Module

M_Authorization: Authorization Module

M_CourseManagement: Course Management Module

M_Enrollment: Enrollment Module

M_Projects: Projects Module

M_StudentsTeams: Students and Teams Module

M_Recommendation: Recommendation Module

M_ApprovalWorkflow: Approval Workflow Module

M_Threads: Threads Module

M_Notifications: Notifications Module

M_Checklist: Checklist Module

M_CourseReport: Course Report Module

M_Persistence: Persistence Module

M_AuditLog: Audit Log Module

Level 1	**Level 2**
Hardware-Hiding Module	M_HardwareHiding
Behaviour-Hiding Module	M_UserInterface M_Authentication M_Authorization M_CourseManagement M_Enrollment M_Projects M_StudentsTeams M_Recommendation M_ApprovalWorkflow M_Threads M_Notifications M_Checklist M_CourseReport
Software Decision Module	M_Persistence M_AuditLog

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

The design decisions made to satisfy the requirements are:

- **Access checks.** Every user signs in through M_Authentication before doing anything, and every action is checked by M_Authorization against the user's role in that course (NFR_Authenticate, NFR_Authorize, FR_AuthorizationRBAC).
- **Supervisor verification as a role.** An unverified external supervisor holds a separate role. Verifying them is a role change in M_Authorization, so all access rules stay in one module (FR_ExternalSupervisorVerification, FR_ExternalSupervisorScope).
- **Course settings as data.** Each course's enabled features and settings are stored as data in M_CourseManagement, not written into the code. This lets a feature be turned off on a live system (FR_DisableFeatures, NFR_LiveDisable).
- **External services hidden by the module that uses them.** Each outside system is reached only through the one module that needs it: identity providers through M_Authentication, the external approval system through M_ApprovalWorkflow, the enrollment source through M_Enrollment, and any service used to deliver notifications through M_Notifications. Replacing an outside system therefore changes one module (FR_MacSSO, FR_ExternalApproval, FR_ImportEnrollment).
- **Stored notifications.** A notification is stored before it is sent, and M_Notifications retries it until delivery is confirmed (FR_DurableNotifications, FR_ActionRequiredEmail).
- **One audit log.** User actions and errors from external services are all recorded through M_AuditLog, which hides how the records are made tamper-evident (NFR_AuditLog, NFR_AuditTamper, FR_LogExternalErrors).
- **One path to storage.** All records are saved and read through M_Persistence, which handles concurrent changes, encryption at rest, and backups (NFR_ConcurrencySafety, NFR_EncryptRest, NFR_Recoverable). All changes resulting from one user action are saved in a single transaction: either all of them are saved or none are. For example, rejecting a team's project and releasing the project slot reserved for that team either both succeed or neither is saved.
- **Passwords never stored in plain text.** Passwords are only stored as hashes, produced inside M_Authentication (NFR_PasswordHash).
- **Encrypted connections.** The user interface and the server talk only over an encrypted (HTTPS) connection, provided by the web framework and operating system behind M_HardwareHiding (NFR_EncryptTransit).

- **Web application.** The user interface runs in the browser and the rest of the system runs on a server (MGUC_WebDelivery), so the system works on any supported browser and screen size (NFR_Browsers, NFR_ScreenSize).

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Keystone* means the module will be implemented by the Keystone software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware-Hiding Module (M_HardwareHiding)

Secrets: The platform the system runs on.

Services: Serves as a virtual hardware used by the rest of the system, so the system can accept input, display output, store files, and make network connections, including encrypted (HTTPS) connections between the browser and the server.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. Changes in the SRS will need to be reflected here.

Implemented By: –

7.2.1 User Interface Module (M_UserInterface)

Secrets: The design of each view.

Services: Shows information to the user and collects their input for every user task.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.2 Authentication Module (M_Authentication)

Secrets: How a user is identified and verified.

Services: Signs users in through McMaster SSO or the provider used by external supervisors, keeps each user's identity (name, email, and linked sign-in account), stores passwords only as hashes and checks passwords against them, and re-verifies users before high-risk actions.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.3 Authorization Module (M_Authorization)

Secrets: What each user is allowed to do.

Services: Records, changes, and looks up each user's role, and decides whether a user may take an action or see a record based on that role. An unverified external supervisor holds a separate role, so verifying them is a role change.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.4 Course Management Module (M_CourseManagement)

Secrets: The setup of each course.

Services: Stores each course's settings and enabled features, keeps track of which features have pending work, and refuses to turn off a feature that has pending work. Lets instructors and TAs publish and update course material, and lets students and supervisors read it.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.5 Enrollment Module (M_Enrollment)

Secrets: How a course's student list is obtained.

Services: Reads an enrollment list from Avenue to Learn or an uploaded file, previews it against the current course list, applies it after confirmation, adds single students, and flags students missing from the latest list.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.6 Projects Module (M_Projects)

Secrets: The project records and their life cycle.

Services: Gives the proposal form for a course and checks that a proposal has every required field. Records proposals and their review, tracks how many teams a project can take, and records team applications, supervisor decisions, and agreements. Reserves and releases project slots, and withdraws competing applications when a team reaches agreement. Returns the projects a user may see, searched, filtered, and sorted as requested.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.7 Students and Teams Module (M_StudentsTeams)

Secrets: How students form teams.

Services: Creates, updates, and shows student and team profiles. Creates teams, handles invitations and joining, records member roles and the assigned TA, and lists students without a team. Stores each course's team rules, checks a team against them, and reports any rule it breaks.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.8 Recommendation Module (M_Recommendation)

Secrets: The method used to match students, teams, and projects.

Services: Suggests teammates, teams, and projects for a student or team.

Implemented By: Keystone

Type of Module: Library

7.2.9 Approval Workflow Module (M_ApprovalWorkflow)

Secrets: The approval process of each course.

Services: Checks a submission before it is sent, sends it to the approvers (in the platform or in an external system such as a GitLab merge request), and records their decision. Handles changes to approved projects without replacing the existing approval until the changes are accepted.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.10 Threads Module (`M_Threads`)

Secrets: The records that discussion threads can be attached to.

Services: Creates threads on records that allow them, adds replies, and lists a record's threads.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.11 Notifications Module (`M_Notifications`)

Secrets: How users are notified of events.

Services: Decides who to notify about each event, writes the message, delivers it through an external service, and keeps it until delivery is confirmed. Runs timed work, such as deadline reminders and retries of undelivered notifications.

Implemented By: Keystone

Type of Module: Abstract Object

7.2.12 Checklist Module (`M_Checklist`)

Secrets: How a user's required actions are worked out.

Services: Lists a user's required actions and whether each one is done.

Implemented By: Keystone

Type of Module: Library

7.2.13 Course Report Module (`M_CourseReport`)

Secrets: The course report format.

Services: Shows instructors and TAs the status of projects, teams, and supervisors in their courses, and exports a report.

Implemented By: Keystone

Type of Module: Library

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Persistence Module (M_Persistence)

Secrets: How records are stored.

Services: Saves, reads, and changes records safely when several users act at once, saving all changes from one user action in a single transaction, keeps records encrypted, and restores them from backups.

Implemented By: Keystone (on PostgreSQL)

Type of Module: Abstract Object

7.3.2 Audit Log Module (M_AuditLog)

Secrets: How audit records are kept.

Services: Records each user action and each error from an external service, and lets authorized users read the records.

Implemented By: Keystone

Type of Module: Abstract Object

8 Traceability Matrix

This section shows three traceability matrices: between the modules and the functional requirements, the non-functional requirements, and the anticipated changes.

Functional Requirement	Modules
FR_SupervisorProposal	M_Projects
FR_ProposalOptionalFields	M_Projects
FR_StudentProposal	M_Projects
FR_ReviewProposal	M_Projects

FR_ViewApprovedProjects	M_Projects
FR_ViewProjectStatus	M_Projects
FR_StudentProfile	M_StudentsTeams
FR_StudentTeams	M_StudentsTeams
FR_ValidateTeamRules	M_StudentsTeams
FR_TeamDiscovery	M_StudentsTeams
FR_TeamProfile	M_StudentsTeams
FR_TeamRecommend	M_Recommendation
FR_ProjectInterest	M_Projects
FR_DeclineTeam	M_Projects
FR_ProjectThreads	M_Threads
FR_ThreadsNotify	M_Notifications
FR_RequestApproval	M_ApprovalWorkflow
FR_ApprovalResponse	M_ApprovalWorkflow
FR_ExternalApproval	M_ApprovalWorkflow
FR_ApprovalPreflightValidate	M_ApprovalWorkflow, M_StudentsTeams
FR_ApprovalRevision	M_ApprovalWorkflow
FR_DisableFeatures	M_CourseManagement
FR_ReferenceMaterial	M_CourseManagement
FR_RequiredActionsList	M_Checklist
FR_ActionRequiredEmail	M_Notifications, M_Checklist
FR_ReadCourseInformation	M_CourseReport
FR_ExportCourseInformation	M_CourseReport
FR_MacSSO	M_Authentication
FR_AuthorizationRBAC	M_Authorization
FR_UpdateRoles	M_Authorization
FR_ExternalSupervisorVerification	M_Authorization, M_Authentication
FR_ExternalSupervisorScope	M_Authorization
FR_LogExternalErrors	M_AuditLog, M_Notifications
FR_RejectComponentDisable	M_CourseManagement
FR_DisplayTimestamps	M_UserInterface, M_Persistence
FR_DurableNotifications	M_Notifications
FR_ReverifyUsers	M_Authentication
FR_ImportEnrollment	M_Enrollment

FR_UpdateEnrollment	M_Enrollment
FR_EnrollmentDiscrepency	M_Enrollment
FR_NeedsGroup	M_StudentsTeams
FR_AssignTA	M_StudentsTeams

Table 2: Trace Between Functional Requirements and Modules

Non-Functional Requirement	Modules
NFR_ScreenSize	M_UserInterface
NFR_UsabilityFirstSuccess	M_UserInterface
NFR_LearnUnassisted	M_UserInterface
NFR_AccessibilityWCAG	M_UserInterface
NFR_PerfTail	M_HardwareHiding, M_Persistence
NFR_Uptime	M_HardwareHiding
NFR_Recoverable	M_Persistence
NFR_PeakUsers	M_HardwareHiding, M_Persistence
NFR_Browsers	M_HardwareHiding, M_UserInterface
NFR_Container	M_HardwareHiding
NFR_DeployUnassisted	M_HardwareHiding
NFR_ChangeEffort	All modules
NFR_LiveDisable	M_CourseManagement
NFR_Authenticate	M_Authentication
NFR_Authorize	M_Authorization
NFR_ConcurrencySafety	M_Persistence
NFR_EssentialPersonal	M_StudentsTeams, M_Authentication
NFR_EncryptTransit	M_HardwareHiding
NFR_EncryptRest	M_Persistence
NFR_PasswordHash	M_Authentication
NFR_AuditLog	M_AuditLog
NFR_AuditTamper	M_AuditLog
NFR_ComplianceAODA	M_UserInterface

Table 3: Trace Between Non-Functional Requirements and Modules

Anticipated Change	Module
AC_Hardware	M_HardwareHiding
AC_UserInterface	M_UserInterface
AC_SignInProcedure	M_Authentication
AC_RolePermissions	M_Authorization
AC_CourseConfig	M_CourseManagement
AC_EnrollmentSource	M_Enrollment
AC_ProposalTemplate	M_Projects
AC_TeamFormation	M_StudentsTeams
AC_Recommendation	M_Recommendation
AC_ApprovalWorkflow	M_ApprovalWorkflow
AC_ThreadTargets	M_Threads
AC_NotificationRules	M_Notifications
AC_RequiredActions	M_Checklist
AC_Report	M_CourseReport
AC_Persistence	M_Persistence
AC_Audit	M_AuditLog

Table 4: Trace Between Anticipated Changes and Modules

Each module maps to an anticipated change.

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

In Figure 1, the dashed box groups the request-handling modules. An arrow into the box means M_UserInterface uses every module in it, and an arrow out of the box means every module in it uses the target. Blue modules are Behaviour-Hiding, orange modules are Software Decision, and the grey module is Hardware-Hiding.

Every Software Decision module is used by at least one Behaviour-Hiding module.

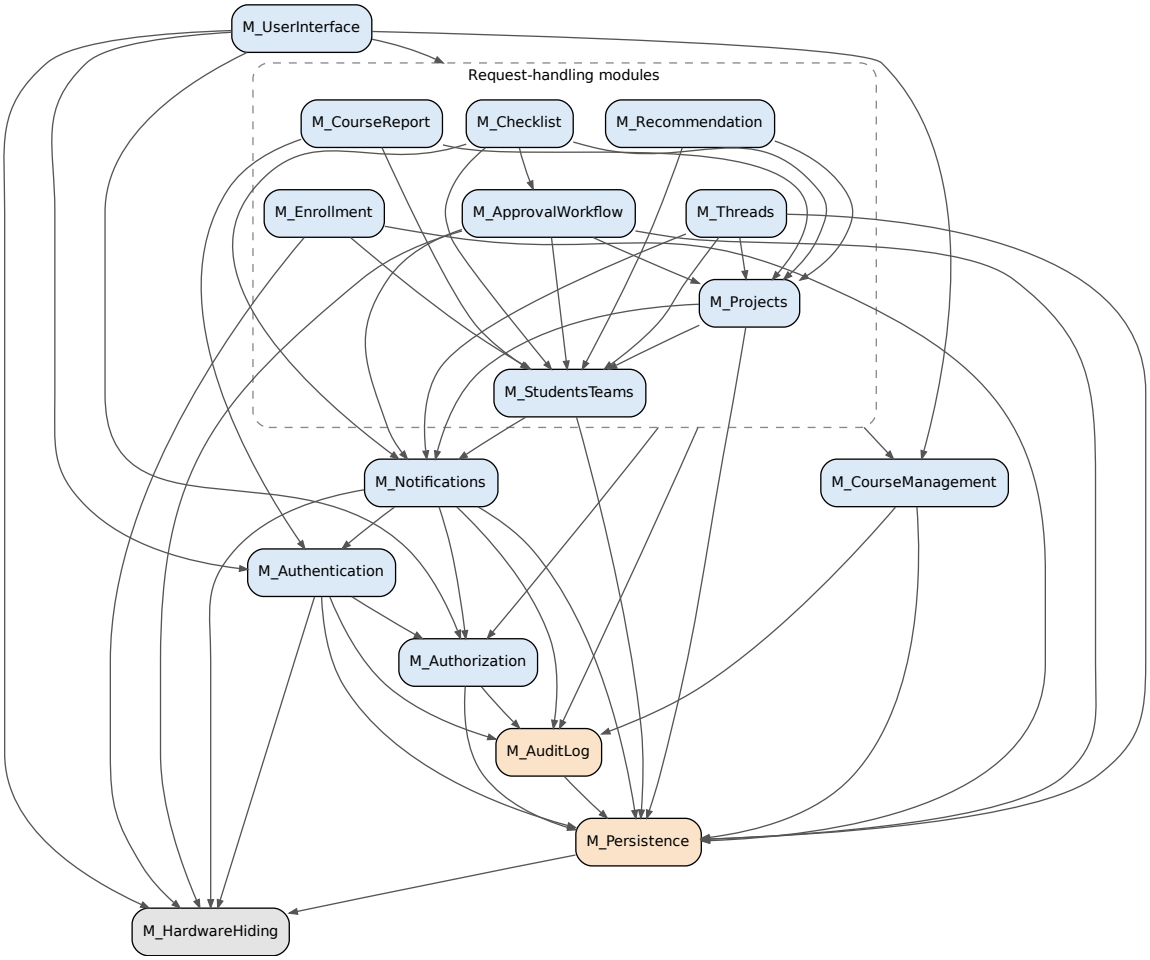


Figure 1: Use hierarchy among modules

10 User Interfaces

...to be completed in Revision 0.

11 Design of Communication Protocols

...to be completed in Revision 0.

12 Database Design

...to be completed in Revision 0.

13 Timeline

...to be completed in Revision 0.

14 Generative AI Use Disclosure

14.1 AI Tools Used

14.2 How and Where Used

14.3 Verification of AI-Generated Content

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.

Ayush Patel, Waqar Ul-Hassan, Ahmed Rashrash, and Andrew Habib. Software requirements specification. 2026. URL https://github.com/Ayushpatel2026/Keystone/blob/main/pdfs/SRS/SRS.pdf.